

LLM 모델 기반 소설 창작 프로젝트

연속성과 일관성을 유지하는 장편 소설 생성 시스템 설계 및 구현

작성자: 한동해
작성일: 2026-01-13

목차

01 프로젝트 개요

- 프로젝트 배경
- 목표 및 방향성
- 핵심 문제 인식

02 주제 선정 이유

- 기존 AI 소설의 한계
- 시스템적 접근 필요성

03 시스템 구조

- 전체 시스템 구조
- 주요 구성 요소
- 파이프라인 설계

04 프롬프트 설계

- 시스템 프롬프트
- 회차별 프롬프트
- 프롬프트 설계 원칙

05 문제점 및 해결

- 중복 발생 문제
- 요약·연결 붕괴
- 개선 방안 적용

06 결론 및 의의

- 주요 성과
- 한계점
- 향후 발전 방향

프로젝트 개요

<https://sgsg.hankyung.com/article/2022031186091>



프로젝트 배경 및 목표

본 프로젝트는 대규모 언어 모델(LLM)을 활용하여 연속성과 일관성을 유지하는 장편 소설 생성 시스템을 설계·구현하는 것을 목표로 함

LLM 기반 소설 창작의 한계

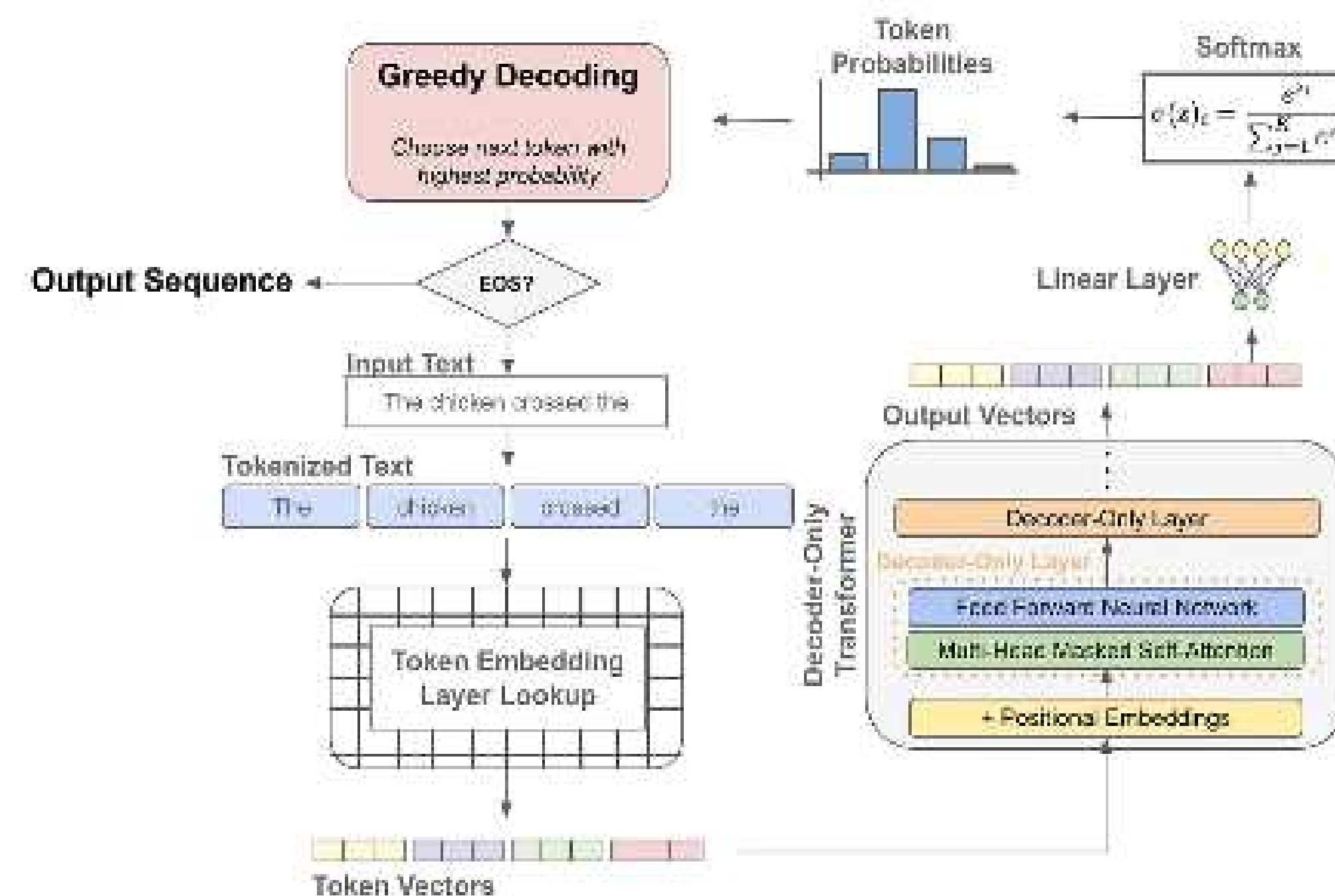
최근 LLM은 다양한 분야에서 활용되고 있으나, 장편 서사와 같이 여러 단계에 걸쳐 맥락을 유지해야 하는 창작 작업에서는 여전히 구조적 한계를 보임

주제 선정 이유



<https://kr.imyfone.com/voice-generator-tips/ai-writing-tools/>

<https://blogs.novita.ai/ko/what-are-large-language-model-settings-temperature-top-p-and-max-tokens/>



기존 AI 소설의 한계

- 단발성 출력에 초점
- 이전 화와 모순되는 전개
- 설정 및 인물 성격 붕괴
- 동일 사건/표현 반복

시스템적 접근의 필요성

- 장편 소설 전체를 하나의 시스템으로 관리
- LLM 출력을 구조적으로 통제
- 안정적 서사 생성을 위한 파이프라인 구축
- 맥락 유지 메커니즘 설계

시스템 전체 구조

Gradio UI

웹 기반 인터페이스로 화별 기획 및 요약 관리

LLM

Ollama 활용 gemma3:4b 모델 로컬 환경 구현

Writer Pipeline

시스템/회차별 프롬프트, 요약, KB 조합 처리

Summarize Pipeline

생성된 챕터 요약 및 다음 화 입력 연결 처리

KB(RAG)

세계관/설정/규칙 관리, ChromaDB 연동 예정

```
novel_maker
├── chapters
├── data
├── scripts
├── src
│   ├── __pycache__
│   ├── pipelines
│   ├── prompts
│   │   ├── chapter.txt
│   │   ├── summarize.txt
│   │   └── system.txt
│   ├── rag
│   ├── utils
│   ├── __init__.py
│   ├── cli.py
│   ├── config.py
│   └── llm_client.py
├── vectordb
├── .env
├── .env.example
├── .gitignore
├── app.py
├── meta.json
├── README.md
└── requirements.txt
```

프롬프트 설계

프롬프트는 LLM 기반 소설 창작의 핵심 요소로, 시스템 프롬프트와 회차별 프롬프트로 구분하여 설계하였습니다.

각 프롬프트는 소설의 일관성과 연속성을 유지하는 데 중요한 역할을 합니다.

시스템 프롬프트의 필요성

- 매 화마다 변하지 않는 '헌법' 역할 수행
- 소설 전체의 일관성 유지를 위한 기반
- 모델의 출력 범위를 제한하는 가이드라인

시스템 프롬프트의 구성 요소

- 주인공의 성격 및 행동 원칙
- 세계관 및 금기 사항
- 문체 및 시점
- 마법/기술 등 설정 규칙

시스템 프롬프트의 효과

- LLM이 자유롭게 창작하되 일정 범위 내 유지
- 인물 성격 붕괴 방지
- 세계관 일관성 확보
- 문체 통일성 유지

프롬프트 설계 및 역할

회차별 프롬프트 구성

- 각 화의 개별 목표와 방향성 설정
- 스토리 전개와 일관성 유지 담당
- 시스템 프롬프트와 달리 회차마다 변경
 - 구체적인 지시사항 포함

중심 사건 설정

- 해당 화에서 다룰 핵심 사건 명시
 - 장면 순서와 전개 방식 지정
- 주요 갈등 요소와 해결 방향 제시
 - 인물 간 상호작용 방향 설정

제약 조건 설정

- '등장하지 말 것' 형태의 부정 제약
- '사용하지 말 것' 형태의 금지 요소
 - 특정 소재나 표현 제한 설정
- 출력 안정성에 큰 영향을 미침

연결성 확보 장치

- 다음 화로 이어지는 '갈고리' 설정
 - 미해결 요소 남기기 전략
- 복선 배치를 위한 지시사항
 - 전체 서사의 연속성 확보

중복 발생 문제

이치로가 빙음의 근원을 찾던 중, 숲의 한쪽에서 일어나는 지진같은 충격이 느껴졌다.

"어... 어라? 지진이야?"

이치로가 지진같은 충격을 감당하려던 중, 숲 속에서 일어나는 폭발음이 들려왔다.

"저건, 뭐지?"

이치로가 폭발음의 근원을 찾던 중, 숲 속에서 튀어나오는 박쥐 떼를 보았다. 박쥐들은

"이건, 또 뭐야?"

이치로가 박쥐들을 막으려던 중, 숲 속에서 들려오는 울음소리가 들려왔다.

이치로가 빙음의 근원을 찾던 중, 숲의 한쪽에서 일어나는 지진같은 충격이 느껴졌다.

"어... 어라? 지진이야?"

이치로가 지진같은 충격을 감당하려던 중, 숲 속에서 일어나는 폭발음이 들려왔다.

"저건, 뭐지?"

이치로가 폭발음의 근원을 찾던 중, 숲 속에서 튀어나오는 박쥐 떼를 보았다. 박쥐들은

"이건, 또 뭐야?"

이치로가 박쥐들을 막으려던 중, 숲 속에서 들려오는 울음소리가 들려왔다.

중복 발생 원인

- LLM의 반복(looping) 성향
- 고정된 프롬프트로 인한 패턴 재귀
- 스트리밍 출력 과정의 반복 감지 부재
- 높은 temperature 값 설정

중복 유형 분석

- 동일 문단 반복
- 유사 사건 중복 전개
- 인물 대사 패턴 반복
- 장면 묘사 중복

중복 문제 해결

```
class NovelWriter:
# 4) chapter template
chapter_tmpl = _load_prompt("chapter.txt")
if not chapter_tmpl:
    # 최소 안전 fallback
    chapter_tmpl = (
        "[이전 화 요약]\n{prev_summary}\n\n"
        "[이번 화 기획/요청]\n{user_brief}\n\n"
        "[작성]\n- 기획서 순서대로만 진행\n- 새 인물/설정 추가 금지\n- 반복 금지\n- 클리프행어로 종료\n\n"
        "[본문]\n"
    )
```

모델 파라미터 최적화

- Temperature 값을 0.7에서 0.6로 낮춰 안정성 확보
- 반복 억제 옵션(repeat_penalty) 값 증가로 다양한 표현 유도

프롬프트 및 파이프라인 개선

- 프롬프트에 '동일 사건 반복 금지' 규칙 명시
- Writer 파이프라인에 반복 감지 및 후처리 로직 추가

```
# 0.8은 즉흥/루프가 잘 생겨 기본값을 안정적으로 낮춤
temperature: float = 0.6

# ✅ 반복 억제/출력 안정화 옵션 (Ollama /api/generate options)
top_p: float = 0.9
top_k: int = 40
repeat_penalty: float = 1.15

def to_options(self) -> Dict[str, object]:
    return {
        "temperature": self.temperature,
        "top_p": self.top_p,
        "top_k": self.top_k,
        "repeat_penalty": self.repeat_penalty,
    }
```

요약·연결 문제

⌚ 요약 생성 중...

- 이치로는 숲에서 농사를 시작한다.
- 숲 깊은 곳에서 거대한 기척을 느낀다.
- 정체불명의 존재와 마주친다.

문제 원인

- 스트리밍 UI용 문구가 요약에 포함됨
- 자동 요약을 무조건 다음 화에 사용
 - 요약 품질 검증 없이 연결에 활용
 - 요약 과정의 정보 손실 발생

몽이: "너의 마음을 다스리는 훈련, 그리고 너의 몸을 단련하는 훈련이다. 또한, 자연과 교감하고, 그 지혜를 배워"

이치로: "알겠습니다. 몽이 선생님의 가르침을 받들어, 훌륭한 영웅이 되겠습니다!"

몽이: "좋다. 그럼, 이제부터 너의 훈련을 시작하자."

(다음 날...)

이치로와 몽이는 매일같이 훈련을 시작했다. 몽이는 이치로에게 검술, 무술, 그리고 자연 지식, 마법의 기초를

하지만, 숲에는 이치로를 노리는 악당이 있었다. 악당은 숲을 파괴하고, 사람들을 괴롭히며, 세상을 혼란에 빠뜨

이치로: "몽이, 악당의 소식이 들려왔습니다. 그는 숲의 정령을 해치고, 숲을 파괴하려고 합니다!"

몽이: "좋다. 너는 이제부터 숲을 지키는 영웅의 역할을 해야 한다. 너의 힘을 사용하여 악당을 막아내야 한다."

발생 현상

- 이전 화와 내용 불일치
- 인물 설정 및 성격 붕괴
- 사건 전개의 논리적 오류 발생
 - 세계관 설정 충돌

파이프라인 개선

```
# 진행 문구/오염 제거용
_PROGRESS_RE = re.compile(r"^\s*🕒\s*요약\s*생성\s*중\.\?\.\?\.\?.*?\n\n", re.MULTILINE)
```

```
# ✅ (1) 이전 화 요약 선택
summary_dir = gr.Textbox(label="요약 폴더 경로", value=SUMMARIES_DIR)
summary_select = gr.Dropdown(
    label="이전 화 요약 선택",
    choices=list_summaries(SUMMARIES_DIR),
    value=None,
    interactive=True,
)
```

```
summary_select.change(
    fn=lambda f, d: load_text_from(d, f),
    inputs=[summary_select, summary_dir],
    outputs=prev_summary,
)
```

요약 텍스트 정제 프로세스

- 저장 전 요약 텍스트 정제(cleaning) 단계 도입
- UI 관련 지시문 자동 필터링 시스템 구현

사용자 통제 강화

- 이전 화 요약을 사용자가 직접 선택하는 UI 개선
- 자동 연결 로직 제거 및 수동 통제 방식 채택

결론 및 피드백

결론

본 프로젝트를 통해 LLM 기반 창작 시스템의 핵심은 단순히 '잘 쓰게 하는 것'이 아니라 '잘못 쓰지 못하게 구조를 설계하는 것'임을 확인했습니다.

- 장편 서사에서는 맥락 유지가 가장 중요한 과제
- 프롬프트 설계, 파이프라인 구조, 입력 데이터 관리가 필수적
- 사용자 통제와 시스템 자동화의 균형이 중요

이러한 구조적 접근을 통해 LLM을 보다 안정적인 창작 도구로 활용할 수 있음을 입증했습니다.

아쉬웠던 점 및 향후 개선 방향

본 프로젝트를 진행하며 가장 크게 느낀 아쉬움은 프롬프트 설계의 완성도가 결과물의 품질을 과도하게 좌우했다는 점입니다.

프롬프트를 충분히 정교하게 작성하지 않을 경우, LLM은 기획 의도와 다른 방향으로 서사를 전개하거나 설정 붕괴, 중복 출력과 같은 문제를 쉽게 발생시켰고 이로 인해 매 화마다 비교적 긴 분량의 프롬프트를 작성해야 했으며, 이는 사용성 측면에서 부담이었습니다.

향후에는 시스템 프롬프트의 역할을 더욱 강화시키고 반복적으로 사용되는 지시를 템플릿화하며 최소 입력으로도 안정적인 출력이 가능하도록 프롬프트 구조를 개선할 필요가 있다고 생각합니다.